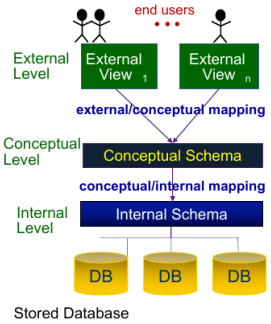
MODULE 0

DBMS facilitates the processes of;
Defining; specifying the types, structures, and constraints for the data
Constructing; process of storing the data on some storage medium controlled by DBMS
Manipulating; querying the DB to retrieve specific data, updating data to reflect changes in UoD
Sharing; allowing multiple users/programs to have access simultaneously

Database System Components
Stored Database; a collection of related information
DBMS; software that defines, constructs, manipulates database
Applications; programs that manipulate the database
Users; people who use the database through DBMS or application interface



Three-Schema Architecture
External Level provides accses to particular parts of the database
Conceptual Level describes the structure of the whole database for a community of users
Internal Level describes the physical storage of the data

Data Independence via Three-Schema Architecture
Data Independence the ability to change the schema at one level of a DB system without having to change the schema at the next higher level
Logical Data Independence ability to change the conceptual schema without changing external views or applications
Physical Data Independence ability to modify the physical schema without changing the conceptual schema

MODULE 1

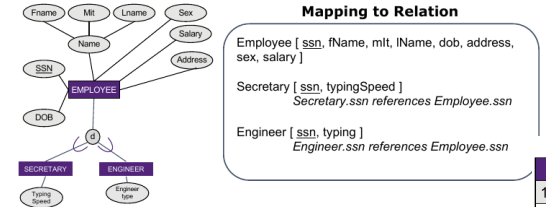
Entity; physical/conceptual object which has data associated with it. Has various attributes associated with it. Same entity can have different attributes depending on the system
Entity Type; provides format for data to be recorded to represent a particular entity. Entity type is described by name and attr.
Entity Sets; the collection of all entities of a particular entity type in the database at any point in time (can be mapped to table)
Key Attributes; a unique identifying attribute of an entity. It is distinct for every entity in the entity set.
Several Attribute Keys; composite key attributes can also be a unique identifyer, only composition of all attributes in composite attribute necessarily needs to be unique
Value Set of Attributes; specify the set of valuse that may be assigned to a particular attribute of an entity (can be NULL)
Composite; an attribute composed of multiple other attributes (not atomic)
Simple; not composite, only one attribute which cannot be divided (atomic)
Multivalued; attribute which ahs multiple values stored within, shown with double-lined ovals
Single; attribute which only has one value
Relationship Types; association among two or more entities, relationship type defines relationship, may have descriptive and key attr.
Relationship Degree; how many entities are involved (Binary 2, Ternary 3, N-ary n (3+))
Entity Roles; participating entites play a particular role in the relationship, role name specifies the role an entityh plays
Recursive Relationship; self-referential relationsihp, an entity can participate more than once under different roles
Relationship Set; collection of relationships of a particular relationship type at any point in time
Existence Dependency; indicates whether the existence of an entity depends on its relationship to another
Weak Entities; entity types that od not have key attributes of their own, must use owners PK, and their partial key to form comp PK
Identifying Relationship; relationship type that relates a eak entity to its owner (entity+rel identified with double lines)
Cardinality Ratio; specifies the number of relationships in that set that an entity can participate in
Relationship Constraints; limit the possible combination of entities that may participate in corresponding relationship set

MODULE 2

Relations; main construct for representing data, a set of records, similar to a table with columns and rows
Domain Types; a set of atomic values (indivisible), each domain has a data type or format
Attributes; each attribute A is the name of a role played by some domain D in a relation R (column of table)
Tuples; each tupple t is an ordered list of n values, known as n-tuple's
Relation Schema; denoted by R[A1,A2,...An], integer n is degree of relation
Relation Instance; denoted by r(R) is a set of n-tuples, r = {t1,t2,tn}

Integrity Constraints (when violation occurs)
Domain Constraint; when attribute's value doesn't appear in corresponding domain
Key Constraint; when a tuple is inserted/modified such that it has the same key value as another tuple
Entity Integrity Constraint; when tuple is inserted/modified such that part of it's PK contains NULL
Referential Integrity Constraint; when trying to insert/modify tuple with FK that doesn't correctly reference PK
Semantic Integrity Constraint; implementation of organisational policies (predefined constraints)
Insertion/modification can cause violate all 5 constraints, Deletion can cause Referential or Semantic

ER to Relational Mapping – (7+1 Steps)
(1) Entity Map; for each entity type, create a relation, choose a key as the PK, include all simple attr.
(2) Weak Entity Map; for each weak entity, create a relation, set its PK as the combination of PK of owner entities and its own partial key, include FK from the relation to the PK of owner entity
(3) 1:1 Relationship; choose one of participating entity types (preference total participation), include FK from chosen entity type back to PK of the relation of the other entity type, include all simple attr. of relation as attributes of relation of the chosen entity type
(4) 1:N Relationship; identify participating entity at N-side of relationship, FK should be PK of entity type on 1-side, include only simple attributes of the relationship type as attributes of the relation on the N side
(5) M:N Relationship; create new relation, include FK from relation to PK of participating entity types, combination of FKs will form PK of R
(6) Multivalued Attr.; create new relation, include FK from relation to PK of associated entity type, PK is combo of multivalued attr and PK of its associated entity type, if multivalued attr is composite, include only simple component
(7) N-Ary Relationship; create new relation, include FK from relation to PK of participating entity types, combo of FKs from entity types with many cardinality will fom PK, R can have its own attributes that contribute to PK, include any simple attributes of R as attributes of the new relation
(8) Super/Subclass; create new relation, PK of each of the subclasses is the PK of the superclass, include FK from relation to PK of relation of its superclass entity type, include all simple attributes
(8) SHOULD BE DONE AFTER (1) OR (2) IF NEEDED



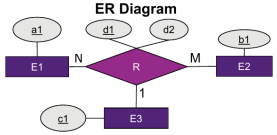
Mapping to Relation

Employee [ssn, fName, mIt, lName, dob, address, sex, salary]

Secretary [ssn, typingSpeed]
Secretary.ssn references Employee.ssn

Engineer [ssn, typing]
Engineer.ssn references Employee.ssn

ER Diagram



Mapping to Relation

E1 [a1] E2 [b1] E3 [c1]
R [a1, b1, d1, c1 d2]
R.a1 references E1.a1
R.b1 references E2.b1
R.c1 references E3.c1

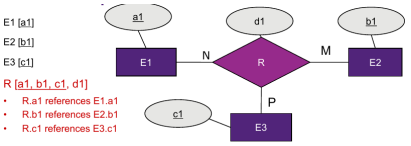
Employee

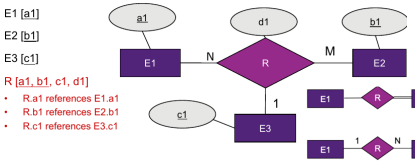
ID	Name	Salary	Department
1751	Paris Lane	60,000	2
4671	Anna Lee	70,000	1
1023	Ben Cho	70,000	4
2034	Jack Smith	40,000	1
2670	Grace Mills	50,000	2

Department

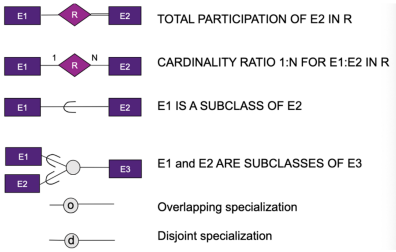
ID	Name	Manager
1	Marketing	4671
2	Development	1751
4	Human Resources	1023

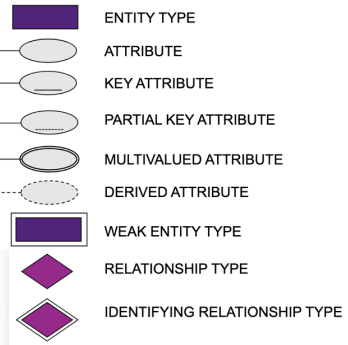
LOL	John Doe	70,000	4	Domain constraints
4671	John Doe	70,000	4	Key constraints
NULL	John Doe	70,000	4	Entity Integrity constraints
1111	John Doe	70,000	6	Referential Integrity constraints
1111	John Doe	99,999	4	Semantic Integrity constraints





Specialisation may be:
- total
- partial





MODULE 3

Aggregation; functions that produce summary of values from a set of tuples, can be used in SELECT or HAVING
Grouping; for considering groups of rows in table, must include any attribute that appears in SELECT or be in aggr fn
HAVING; where clause for GROUP BY
Equi-Join; join condition only has equality comparison
Theta-Join; join condition allows for logical operators to select subset of cartesian product
Inner Join; tuple is included only if matching tuple exists in both relations
LEFT JOIN; a includes all rows from first table
RIGHT JOIN; include all rows from second table
FULL OUTER JOIN; includes all rows from both tables (not implemented in MySQL)
UNION; produces relation that includes all tuples that appear only R1, only R2, or both R1 and R2
INTERSECT; produces relation that includes tuples that appear in both R1 and R2
DIFFERENCE (R1-R2); produces relation that includes all tuples that appear in R1 but do not appear in R2
Non-Correlated Subqu; results returned from inner query to outer clause, evaluated from inside out, outer query takes action based on results of inner query
Correlated Subquery; conditions in subquery WHERE clause have references to some attributes of a relation in the outer query, outer SQL statement provides the valuse for the inner subquery to use, evaluated once for each combo of tuple in outer query
Subqueries can return SETS or SINGLE VALUES

Division can be useful for answering queries which include FOR ALL or FOR EVERY
R1/R2 results in relation containing columns in R1 but not in R2. Resulting relation contains tuples t, such that a value in t appears in R1, in combination with every tuple in R2.

R1 and R2 must be division compatible (i.e. (last) n columns of R1 must be identically named to columns in R2, where n is the degree of R2)

Example; "find movie stars who acted in at least all the movies produced in the year 1934" ==
(StarsIn)/(IDs from Movies in 1934)

A

sno	pno
s1	p1
s1	p2
s1	p3
s1	p4
s2	p1
s2	p2
s3	p2
s4	p2
s4	p4

B1

pno
p2

B2

pno
p2
p4

B3

pno
p1
p2
p4

A/B1

sno
s1
s2
s3
s4

A/B2

sno
s1
s4

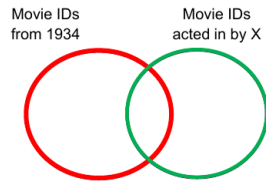
A/B3

sno
s1

Query

```
SELECT starID, name
FROM MovieStar X
WHERE NOT EXISTS (
    SELECT *
    FROM StarsIn S
    JOIN Movie M ON S.movieID = M.movieID
    WHERE M.year = 1934
        AND M.movieID NOT IN (
            SELECT movieID
            FROM StarsIn M2
            WHERE M2.starID = X.starID
        )
)
```

Conceptual



MovieStar (X)

starID	name
1	Grace
2	Jack
3	Juliet

StarsIn JOIN Movie

starID	movieID	year	...		
1	100	1932	2	400	1950
1	200	1934	1	400	1950
2	300	1934	3	200	1934
2	200	1934	3	300	1934

StarsIn (M2)

movieID
200
300

BCNF Decomposition

(1) Check if FDs are BCNF
(2) For each FD X→Y, split into R1 = {X}+, R2 = R – {X}+ + X
(2) For each FD X→Y, split into R1[R-Y] and R2[X U Y]
(3) For each subrelation, check if all of its FDs are now BCNF
    If they are, we keep sub-relation as it is.
    If not, we repeat Step 2 for sub-relations that do not satisfy BCNF

MODULE 4

Functional Dependencies
A functional dependency X→Y holds on R
If for every legal instance of R such as r, for all tuples t1, t2;
If t1[X] = t2[X] → t1[Y] = t2[Y]

Superkey; a set of attributes such that no two tuples have the same values
(Candidate) Key; a minimal superkey where none of the attributes can be removed to make another superkey
Primary Key; one of the selected candidate keys
Foreign Key; a reference to a primary key from another table
Prime Attributes; an attribute in any candidate key
Non-Prime Attributes; an attribute that is not a member of any candidate key

Finding Keys
Given a relation R and associated set of FDs F,
For any subset S of attributes R, S is a key iif
(1) S+ = R, (2) there is no such S' ⊂ S such that S'+ = R.

Closure of X (X+)
The set of attributes determined by X under F.
(1) X+ initialy contains all attributes of X
(2) For each FD in F, if the LHS of of the FD is a subset of X+, add the RHS to X+
(3) If step 2 resuletd in changes to X+ then repeat 2, otherwise finish

Closure of F (F+)
Given a set of FDs many implicit FDs can be derived. F+ includes impl + expl FDs
Trivial FDs hold regardless of F (i.e. LHS conatins RHS)
Non-trivial FDs depend on a given F

Transitive Dependency
X→Y in a relation R is a transitive dependency iff there exists a set of attributes Z in R such that; Z is neither a candidate key nor a subset of any key of R, and both X→Z and Z→Y hold.

Partial Dependency
X→Y is a partial dependency if some attribute A can be removed and the dependency still holds, (X – {A}) → Y

For a relation R with FD set F, for any non-trivial X→A in F+
1NF – identifying non-atomic values from relations, relations should have no multivalued attributes or nested relations

2NF – identifying partial dependencies (no partial)
X is not a proper subset of of a candidate key for R, or A is prime attribute

3NF – identifying partial and transitive dependencies (no transitive)
X is a superkey for R, or A is a prime attribute

BCNF – identifying all anomalies (at the cost of not preserving all FDs)
X is a superkey for R

BCNF Decomposition
While there exists a relation Q in R that violates BCNF (LHS is not superkey){
Find one FD X→Y that violates VCNF
Replace Q by Q1 [Q-Y] and Q2 [X ∪ Y]}

Determining which FDs apply
For FD X→Y, if decomposed relation S contains {X∪Y} and Y ⊂ X+ then FD holds

3NF Synthesis
S = {}, compute the minimal cover G of F, combine all FD w/ same LHS into one
For each X→Y in G{if(no relation in S contains X∪Y){add relation X∪Y to S}}
if(any candidate key is missing from the relations){add relation w/ all prime att.}
Eliminate redundant relations from S

Minimal Cover
G is a minimal cover if every FD is "as small as possible" to get same closure of F
(1) RHS Simplification – every FD only has one attribute on RHS
(2) LHS Simplification – remove redundant attributes from LHS
For X→A in F, where X has multiple attributes, if (X-{B})+ = X+ in F, drop B
For AB→C , if {A}+ contains {B}+, we can remove B to get A→C
(3) FD Set Simplification – delete redundant FDs
FD is redundant if LHS closure without considering FD contains RHS

Dependency Preservation
Given a relation R decomposed into R1,…,Rn, F is decomposed into F1,…,Fn
Fi contains all FDs in F with the attributes completely in Ri
R is dependency preserving iff (F1 U … U Fn)+ = F+

MODULE 5

Threats to Database Security (C.I.A principle), LOSS OF;
***Loss of* Confidentiality**; unauthorized disclosure of confidentiality
***Loss of* Integrity**; improper modification to information
***Loss of* Availability**; legitimate user cannot access data objects

Security; concerns how to control the access for data, available for use permitted access
Privacy; concerns how the data can be used, about the ability of individuals to control their data
Security and privacy are closely related but different.
There is usually a tradeoff between accessibility and security.

Database Control Measures
Access Control (DAC/MAC/RBAC); provide access only to users who have right access authority
Inference Control; ensure information about individuals cannot be accessed (typically statistical DBs)
Flow Control; prevent information from flowing to unauthorized users
Data Encryption; used to protect sensitive transmitted data

Discretionary Access Control (DAC) based on granting and revoking privileges, two levels of privileges
Account Level; privileges specified for each account, independent of relations in the account
Relation Level; privileges specified for each relation or view, can be specified at attr level too

WITH GRANT OPTIONS; allows user granted permissions to grant other users
CASCADE; cascade is default for removing privileges (removes privileges from other users down chain)

CREATE/DROP ROLE manager;
GRANT privilegeName ON objectName TO {username/roleName} [WITH GRANT OPTION]
REVOKE privilegeName ON objectName FROM {username/roleName}

Mandatory Access Control (MAC) classifies data (objects) and users (subjects) with security classes

Bell-Lapadula Model
Simple Security Property; subject can't read information from an object that has higher sensitivity label than the subject, known as no read up or NRU

Star Property; subject cannot write information to an object that has lower sensitivity label thanthe subject, known as no write down or NWD

Discretionary policies have a higher degree of flexibility, but don't impose control on how information is propagated. Mandatory policies ensure high degree of protection, rigid but prevent information flow.

Role-Based Access Control (RBAC) permissions associated with organisational roles, can use w DAC/MAC

Flow Control; regulates distribution or flow of information among accessible objects
Flow Policy; specifies channels along which info can move: C and N, prohibit flow from C to N

SQL Injection
SQL Manipulation; changes in SQL command in the application (e.g. adding conditions to WHERE)
Code Injection;add additoinal SQL statements that are then processed
Function Call Injection; DB or OS function call inserted into vulnerable SQL to manipulate data or RCE

SQL Injection Risks
Database Fingerprinting; attacker determines type of database being used
Denial of Service; attacker denies service to valid user(s)
Bypassing Authentication; attacker gains accses to database without valid authorization
Identifying Injectable Parameters;attacker determines information about backend structure
Executing Remote Commands; attacker executes harmful commands remotely
Performing Privilege Escalation; attacker upgrades their access level

Protection Techniques
Bind Variables using parameterised statements, protects against injection, improves performance
Filtering Input using input validation, remove escape characters from input strings
Function Security standard and custom functions should be restricted